AOSP 进阶丛书 · 第十九册 · ANDROID 13 (API 33)

Telephony 电话栈精读

RIL(:108) socket 模型、GsmCdmaPhone(:141) 状态机、拨打电话七跳链、APN 数据连接汇入网络栈、IMS/VoLTE 架构。

源码基线：AOSP Android 13 (TP1A · SDK 33) · 行号全部实测核验

目录

前言：打电话的那个世界 2
第一章 Telephony 架构总览 2
第二章 RIL：与 Modem 的抽象层 3
第三章 GsmCdmaPhone：电话状态机 3
第四章 数据连接与 APN 4
第五章 IMS 与 VoLTE 4
第六章 设计决策五问 4
附录 A 观测 5
附录 B 快问快答（12 条） 5
附录 C 自检题 6
第七章 RIL 请求管线：SerialNumber 配对与响应分发 6
第八章 开机注册链：从 PhoneFactory 到 IRadio HAL 绑定 11
第九章 网络注册状态机：ServiceStateTracker 的双引擎 16
第十章 呼叫建立链：从 Telecom 到 CLCC 轮询 20
教科书习题集（分级） 24
术语表 26
排障手册：Telephony 电话栈问题定位 28
Telephony 核心速查表 30

本册速览

RIL.java:108（Radio Interface Layer——与 modem 的抽象层）
GsmCdmaPhone：电话状态机（SIM/信号/呼叫三合一）
Telephony framework：frameworks/opt/telephony/（非 frameworks/base！）
IMS/VoLTE：IP 多媒体子系统
APN 与数据连接：DataNetworkController

前言：打电话的那个世界

Telephony 是 Android 中**最古老也最独立**的子系统：RIL（Radio Interface Layer）与 modem 对话、GsmCdmaPhone 管状态机、IMS 走 VoLTE——它们的源码住在 `frameworks/opt/telephony/`（不在 frameworks/base！）而非核心服务。本册讲清这套独立王国的架构。

第一章 Telephony 架构总览

> **设计洞察 · 学习目标**
> 1. 画出 Telephony 三层结构并标注每层源码目录；2. 说出 RIL/GsmCdmaPhone/PhoneFactory 的职责与行号锚点；3. 解释 Telephony 为何住在 frameworks/opt 而非 frameworks/base；4. 说出 phone 进程（com.android.phone）与 TelephonyProvider 数据库的分工

text Telephony 三层（读码归纳）

```
App 层：Dialer/Contacts/SMS（packages/apps/）
→ TelephonyManager（公共 API，frameworks/base）
Framework 层：frameworks/opt/telephony/（独立于 frameworks/base！）
├── Phone.java（抽象基类）
├── GsmCdmaPhone.java:141（2G/3G 状态机）
├── RIL.java:108（Radio Interface Layer——与 modem 对话）
├── PhoneFactory.java:66（Phone 实例工厂）
├── DataNetworkController（APN 数据连接）
└── ImsPhone（VoLTE/IP 电话）
服务层：TelephonyProvider（telephony.db：APN/SMS 数据库）
+ phone 进程（com.android.phone）
```

先修要求与学习路线

先修：已读第一册（架构总览）。

读完本书你将能够：

- 解释Telephony 电话栈的核心机制与设计决策
- 追踪关键调用链并标注每次跨进程跳
- 使用 dumpsys/日志定位本领域问题
- 回答书7 对应章节的面试题

> **设计洞察 · 逐段精读：为什么 Telephony 在 frameworks/opt 而非 frameworks/base**
> 历史（最初是独立项目）+ 演进节奏不同（telephony 每版大改而 core 稳定）+ **Mainline 化的前置**（telephony 已迁入 packages/modules/Connectivity 附近的独立 APEX）——独立目录让模块化过渡更平滑。

第二章 RIL：与 Modem 的抽象层

设计洞察 · 学习目标

1. 说出 RIL 在三层结构中的位置与 CommandsInterface 的角色；2. 描述 socket 通路的历史设计与 Treble 化后的 HAL Binder 通道（详见第八章）；3. 列出 RIL 对 modem 的命令域（通话/短信/网络/数据/SIM）；4. 区分 Solicited（请求-应答）与 Unsolicited（主动上报）两类消息

RIL.java（:108 extends BaseCommands implements CommandsInterface）是 Java 框架与 modem 硬件之间**唯一的桥**：

java frameworks/opt/telephony/…/RIL.java:108 起（骨架+讲解注释）

```
public class RIL extends BaseCommands implements CommandsInterface {
// 讲解注释①：CommandsInterface 定义与 modem 的全部命令：
// dial/hangup/sendSMS/getSignalStrength/setRadioPower…
// → 经 RILSender（Handler 线程）写 socket
// 讲解注释②：rild（native 守护进程，hardware/ril/）接收
// → 厂商 AT/OEM 协议转发 modem
// 讲解注释③：modem 回复 → rild → RILReceiver（另一个 Handler 线程）
// → 解析 → 回调 Phone 状态机
}
```

设计洞察 · 逐段精读：为什么走 socket 而非 Binder

RIL 是 2009 年的设计（Binder 尚未稳定于 native）；且 rild 是 C 守护进程（非 Java），socket 更自然。这解释了 telephony 的"古老感"——它比 Binder 体系更早定型。

第三章 GsmCdmaPhone：电话状态机

设计洞察 · 学习目标

1. 说出 GsmCdmaPhone 封装的状态清单（SIM/信号/呼叫/注册态）；2. 走通七跳拨号链并标注每次 IPC 类型（Binder/socket）；3. 说出 PhoneFactory 按 modem 类型创建实例的规则与 GsmCdmaPhone 合一的动机；4. 知道本章七跳全景与第十章逐行走读的呼应关系

GsmCdmaPhone（:141 extends Phone）封装了**一部手机的全部电话状态**：SIM 卡状态/信号强度/呼叫列表/网络注册态。PhoneFactory（:66）根据 modem 类型创建实例（GSM 或 CDMA，13 已合一为 GsmCdmaPhone）。

text 拨打一个电话的链（读码归纳）

```
Dialer App → TelephonyManager.placeCall(uri)
→ TelecomManager.placeCall（Telecom 系统——phone 进程外的独立服务）
→ ConnectionService（telecom framework）
→ GsmCdmaPhone.dial(number)
→ RIL.dial(number) → socket → rild → modem
→ modem 建立通话 → 回调 RIL → CallTracker 更新 Call 状态
```

```
→ 通知 TelecomManager → InCallUI 显示通话界面
// 讲解注释：七跳，其中 TelecomManager→ConnectionService
// 是 Binder；RIL→rild 是 socket。
```

第四章 数据连接与 APN

设计洞察 · 学习目标

1. 解释 APN 的作用与类型（default/mms/ims/supl 等）；2. 说出 TelephonyProvider→ApnSetting→setupDataCall→NetworkAgent 的完整链路；3. 理解 telephony 是 ConnectivityService（书10 网络评分体系）的供给方；4. 说出 DcTracker（dataconnection/DcTracker.java）与 DataConnection 状态机的分工

DataNetworkController 管理 APN（Access Point Name）数据连接：每个 APN 声明类型（default/mms/ims/supl/…）与认证参数；DataConnection 是一次 PPP/DataCall 会话。

text APN 的完整链

```
TelephonyProvider（telephony.db）预置 APN 列表
→ ApnSetting 解析 → DataNetworkController 匹配
→ RIL.setupDataCall → modem 建立 → 网络可用
→ ConnectivityService（书10 §二）收到 NetworkAgent
——telephony 是 NetworkFactory 的一个供给方！
// 讲解注释：5G/4G 数据链走完 telephony 内部后，
// 终归汇入书10 的网络评分体系——telephony 不独立管路由。
```

第五章 IMS 与 VoLTE

设计洞察 · 学习目标

1. 说出 IMS 与 CS 域（电路交换）的本质区别；2. 解释"运营商 VoLTE 开通=IMS 注册成功"的判定依据；3. 说出 SIP 信令与 RTP 语音流的分工；4. 理解 ImsPhone 与 GsmCdmaPhone 并列挂在 Phone 上（getImsPhone）的结构

IMS（IP Multimedia Subsystem）= 用 IP 网络打电话（VoLTE/视频通话）：ImsPhone 实现 Phone 接口但走 IMS 栈（SIP 信令→RTP 语音）；**语音从 CS 域（电路交换）迁移到 PS 域（包交换）**——打电话不再占"电话信道"而是走数据网。运营商 VoLTE 开通=IMS 注册成功。

第六章 设计决策五问

设计洞察 · 学习目标

1. 复述五问五答的核心结论；2. 用"历史起源+演进节奏+模块化前置"三段论解释 opt 独立目录；3. 说出 socket 设计的由来与 Treble 化（Android 8 起 IRadio HAL Binder）的通道变化；4. 把每一问对应到本册具体章节的源码证据

- **为什么 Telephony 在 frameworks/opt 独立目录？** 历史起源+演进节奏不同+Mainline 化前置（第一章）。
- **为什么 RIL 走 socket 不走 Binder？** 比 Binder 更早定型；rild 是 C 守护——socket 最自然。
- **为什么 GsmCdmaPhone 合一？** CDMA 已淘汰、代码 90% 相同——合并降低维护成本（13 完成）。
- **为什么 TelecomManager 在 telephony 之外？** 统一管理电话/视频/SIP 多种 ConnectionService（telephony 只是其中一种）——**调度权独立于供给方**。
- **为什么打电话不直接调 RIL 还要过 Telecom？** Telecom 管通话策略（蓝牙耳机接听/呼叫等待/多路并发）——App 不应该知道这些细节。

附录 A 观测

目标	命令	看点
电话状态	dumpsys telephony.registry	信号/呼叫/SIM
Telecom	dumpsys telecom	通话列表/连接服务
RIL 日志	logcat -s RILJ RILC	与 modem 的对话
APN	dumpsys telephony.data	APN 配置/数据连接状态
IMS	dumpsys ims	注册状态/通话统计

附录 B 快问快答（12 条）

- Telephony 源码在哪？——frameworks/opt/telephony/（不在 frameworks/base）。
- RIL 是什么？——Radio Interface Layer（:108，与 modem 的抽象层）。
- RIL 走什么 IPC？——socket（比 Binder 更早的设计）。
- GsmCdmaPhone 是什么？——2G/3G/4G 电话状态机（:141）。
- PhoneFactory 干什么？——根据 modem 类型创建 Phone 实例（:66）。
- TelecomManager 管什么？——通话调度（蓝牙/呼叫等待/多路）。
- 拨打电话几跳？——七跳（App→Telecom→Phone→RIL→rild→modem→回）。
- APN 是什么？——Access Point Name（数据连接接入点配置）。
- VoLTE 走什么？——IMS（IP 域，SIP+RTP）。
- 数据连接怎么汇入网络栈？——telephony 是 NetworkFactory 供给方（书10）。
- 5G NSA 与 SA 区别？——非独立（4G 锚+5G 面）vs 独立（纯 5G 核心网）。
- Telephony 在 Mainline 化吗？——已部分迁入独立 APEX（进行中）。

附录 C 自检题

- 从 Dialer 点击到通话建立的全链（七跳含 IPC 类型）。
- RIL 的 socket 模型与 Binder 模型各自适用场景。
- APN→DataConnection→CS（书10）NetworkAgent 的汇入链。

第七章 RIL 请求管线：SerialNumber 配对与响应分发

设计洞察 · 学习目标

1. 说出 RILRequest 的复用池模型与 SerialNumber 的生成规则；2. 走通一条请求从 obtain 入队到 processResponseDone 回调的完整生命周期；3. 区分 EVENT_WAKE_LOCK_TIMEOUT 与 EVENT_BLOCKING_RESPONSE_TIMEOUT 两种超时兜底；4. 梳理 unsolicited 消息从 HAL Indication 回调经 RegistrantList 到状态机的分发链

链路位置：第二章建立了 RIL 的"桥"视角，本章进入桥的内部。RIL 请求管线是整部电话栈的心脏：第九章 pollState 的 getOperator、第十章拨号的 mCi.dial、第四章数据的 setupDataCall——所有对 modem 的操作都汇入同一条管线。理解了它，其余章节里"RIL 发出请求"六个字就都有了微观结构。



图 1 RIL 请求管线：下行 serial 配对记账，上行响应与 unsol 两条来路在 Tracker 消息分发表合并

7.1 RILRequest：请求对象的复用池

每一条发给 modem 的命令在 Java 侧都是一个 RILRequest 对象（`frameworks/opt/telephony/…/RILRequest.java:36`）。它只有十来个字段，却承载了两个关键设计：**SerialNumber 全局递增**（与 HAL 应答一一配对）与**对象池复用**（MAX_POOL_SIZE=4 的单链表池）。

字段	含义
mSerial	进程级全局递增序号，与 HAL 应答配对的主键
mRequest	RILConstants 里的 RIL_REQUEST_* 命令号
mResult	请求完成时要 sendToTarget 的 Message（回调目标）
mStartTimeMs	SystemClock.elapsedRealtime() 打点，用于延迟统计
mWakeLockType	持锁类型（FOR_WAKELOCK/FOR_ACK_WAKELOCK）

java frameworks/opt/telephony/…/RILRequest.java:36-113（骨架+讲解注释）

```
public class RILRequest {
// 讲解注释①：进程级原子序号——从 0 递增，wrap 到 Integer.MAX_VALUE 归零
static AtomicInteger sNextSerial = new AtomicInteger(0);
// 讲解注释②：对象池头节点与同步锁——最多缓存 MAX_POOL_SIZE=4 个
private static Object sPoolSync = new Object();
private static RILRequest sPool = null;
private static final int MAX_POOL_SIZE = 4;
// 讲解注释③：五个核心字段——serial（配对主键）、request（命令号）、
// result（完成回调 Message）、mStartTimeMs（延迟打点）、
// mWakeLockType（持锁标记，决定响应时释放哪把锁）
@UnsupportedAppUsage
int mSerial;
@UnsupportedAppUsage
int mRequest;
@UnsupportedAppUsage
Message mResult;
// time in ms when RIL request was made
long mStartTimeMs;
int mWakeLockType;
...
private static RILRequest obtain(int request, Message result) {
RILRequest rr = null;
synchronized (sPoolSync) {
if (sPool != null) {
rr = sPool; // 讲解注释④：从池头摘一个复用对象
sPool = rr.mNext;
rr.mNext = null;
sPoolSize--;
}
}
if (rr == null) {
rr = new RILRequest(); // 讲解注释⑤：池空才真正 new——
} // 电话请求频次不高，4 个足够周转
// Increment serial number. Wrap to 0 when reaching Integer.MAX_VALUE.
```

```
rr.mSerial = sNextSerial.getAndUpdate(n -> ((n + 1) % Integer.MAX_VALUE));
// 讲解注释⑥：serial 在 obtain 时刻就确定——后续入账本
//  （addRequest）与配对（processResponse）全靠它，中途不可改
rr.mRequest = request;
rr.mResult = result;
rr.mWakeLockType = RIL.INVALID_WAKELOCK;
rr.mWorkSource = null;
rr.mStartTimeMs = SystemClock.elapsedRealtime();
// 讲解注释⑦：result Message 必须带 target（Handler）——否则响应
// 无处可投，这里提前抛 NPE 而不是等响应时才暴露
if (result != null && result.getTarget() == null) {
throw new NullPointerException("Message target must not be null");
}
return rr;
}
```

设计洞察 · 逐段精读：为什么 SerialNumber 从 0 全局递增

serial 随请求写给 modem、又在应答里原样带回——4 字节小整数最省信令；严格递增保证"同型请求并发多路"（例如两个 pollState 同时在途）也能各自配对；getAndUpdate 原子自增还暴露了一个事实——管线必然多线程，所有 Phone 状态机都可能同时往 RIL 塞请求，所以 mRequestList 的每次读写都包在 synchronized 块里。

7.2 入队与出队：mRequestList 账本两端

RIL.java 持有一张"在途请求表" mRequestList（SparseArray，:182），入队在发送前（:1163），出队在响应到达时（:5161）——两端以 serial 为钥匙对称操作：

java frameworks/opt/telephony/…/RIL.java:1163-1178、5161-5196（配对两端+讲解注释）

```
// ===== 发送端：请求入账本（RIL.java:1163）=====
private void addRequest(RILRequest rr) {
acquireWakeLock(rr, FOR_WAKELOCK);
// 讲解注释①：先持 WakeLock——防止 AP 深睡后 modem 应答没人收
synchronized (mRequestList) {
rr.mStartTimeMs = SystemClock.elapsedRealtime();
mRequestList.append(rr.mSerial, rr);
// 讲解注释②：以 serial 为 key 入表——这是"在途请求"的权威账本
}
}
private RILRequest obtainRequest(int request, Message result, WorkSource workSource) {
RILRequest rr = RILRequest.obtain(request, result, workSource);
addRequest(rr);
// 讲解注释③：obtain（领号）+addRequest（入账）二合一——
// 每个 public API（dial/getOperator/…）的第一步都是它
return rr;
}
// ===== 响应端：HAL 回调后按 serial 配对（RIL.java:5161）=====
private RILRequest processResponseInternal(int service, int serial, int error, int type) {
RILRequest rr;
if (type == RadioResponseType.SOLICITED_ACK) {
// 讲解注释④：SOLICITED_ACK 只是"确认收到"，请求仍未完成——
// 只释放 ack 锁，不摘账本
```

```
synchronized (mRequestList) { rr = mRequestList.get(serial); }
if (rr == null) {
Rlog.w(RILJ_LOG_TAG, "Unexpected solicited ack response! sn: " + serial);
} else {
decrementWakeLock(rr);
}
return rr;
}
rr = findAndRemoveRequestFromList(serial);
// 讲解注释⑤（:5183）：最终响应到达——从账本摘除，配对成功
if (rr == null) {
Rlog.e(RILJ_LOG_TAG, "processResponse: Unexpected response! serial: " + serial
+ " ,error: " + error);
return null;
// 讲解注释⑥：查无此 serial=过期/重复应答——直接丢弃，
// 绝不投递给上层状态机（防脏消息）
}
// Time logging for RIL command and storing it in TelephonyHistogram.
addToRilHistogram(rr);
// 讲解注释⑦：请求耗时入 5 桶直方图——RIL 性能的长期观测数据
...
}
```

配对成功后，真正的投递发生在 processResponseDone（:5266）：把 HAL 解析好的返回值打包成 AsyncResult 塞进 rr.mResult 并 sendToTarget——请求对象的生命就此终结（release 归还对象池，RILRequest.java:185）。

7.3 超时兜底：RilHandler 的两道保险

modem 不应答怎么办？RIL 内部的 RilHandler（:289 起）挂了两道计时器：

java frameworks/opt/telephony/…/RIL.java:292-320、331-349（超时兜底+讲解注释）

```
public class RilHandler extends Handler {
@Override
public void handleMessage(Message msg) {
switch (msg.what) {
case EVENT_WAKE_LOCK_TIMEOUT:
// Haven't heard back from the last request. Assume we're
// not getting a response and release the wake lock.
synchronized (mRequestList) {
if (msg.arg1 == mWlSequenceNum && clearWakeLock(FOR_WAKELOCK)) {
// 讲解注释①：mWlSequenceNum 比对——若期间有新请求
// 已为锁"续期"，这枚旧计时器就不清锁
if (mRadioBugDetector != null) {
mRadioBugDetector.processWakelockTimeout();
}
// 讲解注释②：WakeLock 超时是 modem 健康度的负面信号，
// 计入 RadioBugDetector 供 radio bug 检测
if (RILJ_LOGD) {
int count = mRequestList.size();
Rlog.d(RILJ_LOG_TAG, "WAKE_LOCK_TIMEOUT " +
" mRequestList=" + count);
for (int i = 0; i < count; i++) {
```

```
rr = mRequestList.valueAt(i);
Rlog.d(RILJ_LOG_TAG, i + ": [" + rr.mSerial + "] "
+ RILUtils.requestToString(rr.mRequest));
}
// 讲解注释③：打印全部在途请求清单——60 秒无应答后
// logcat -b radio 里最扎眼的一段，排障第一现场
}
}
}
break;
case EVENT_BLOCKING_RESPONSE_TIMEOUT:
int serial = (int) msg.obj;
rr = findAndRemoveRequestFromList(serial);
// 讲解注释④：阻塞式调用的兜底——从账本摘除，不再等
if (rr == null) {
break;
}
// Build a response if expected
if (rr.mResult != null) {
Object timeoutResponse = getResponseForTimedOutRILRequest(rr);
AsyncResult.forMessage(rr.mResult, timeoutResponse, null);
rr.mResult.sendToTarget();
// 讲解注释⑤：构造"超时假响应"（合成默认值）投递——
// 保证调用方不被无限阻塞
mMetrics.writeOnRilTimeoutResponse(mPhoneId, rr.mSerial, rr.mRequest);
}
decrementWakeLock(rr);
rr.release();
break;
}
}
}
```

两道保险的分工：**EVENT_WAKE_LOCK_TIMEOUT**（默认 60 秒，:128 DEFAULT_WAKE_LOCK_TIMEOUT_MS）只清锁不打断——账本特意保留（注释原文：Note: Keep mRequestList），迟到的响应来了仍能配对；**EVENT_BLOCKING_RESPONSE_TIMEOUT**（2 秒）针对阻塞式调用，直接构造合成响应解除阻塞。

7.4 Unsolicited 分发：Indication 类与 RegistrantList

请求-应答走 Response 类，modem 主动上报走 Indication 类。以网络状态变化为例（第九章的主角）：

java frameworks/opt/telephony/…/NetworkIndication.java:238-245（真实代码+讲解注释）

```
public void networkStateChanged(int indicationType) {
// 讲解注释①：NetworkIndication extends IRadioNetworkIndication.Stub——
// modem 侧上报时 HAL Binder 直接调进本方法（oneway 回调）
mRil.processIndication(RIL.NETWORK_SERVICE, indicationType);
// 讲解注释②：校验 indicationType 并维护 wakelock 统计
if (RIL.RILJ_LOGD) mRil.unsljLog(RIL_UNSOL_RESPONSE_NETWORK_STATE_CHANGED);
// 讲解注释③：logcat -b radio 里 "RILJ: UNSOL_NETWORK_STATE_CHANGED" 的出处
mRil.mNetworkStateRegistrants.notifyRegistrants();
// 讲解注释④：广播给全部订阅者——ServiceStateTracker（第九章）
```

```
// 就在此名单上，收到后触发 pollState
}
```

订阅机制在 BaseCommands（RIL 的父类，:108 所述 BaseCommands）里：十几个 RegistrantList（mNetworkStateRegistrants/mCallStateRegistrants/mRadioStateRegistrants…）各管一类上报，状态机构造时调 registerForXxx(this, what, obj) 注册 Handler 消息，unsol 到达即 notifyRegistrants 逐个 sendMessage——经典观察者模式，一次上报可同时驱动多个订阅者。

> **设计洞察 · 设计洞察：SOLICITED 与 UNSOLICITED 是两个平行宇宙**
> 请求响应走 Response 类——有 serial、靠配对、发完即摘账本；主动上报走 Indication 类——无 serial、广播给 RegistrantList、没有"谁发的"概念。RIL 构造函数（:1029 起）成对 new 出 DataResponse/DataIndication、NetworkResponse/NetworkIndication 等 7 组回调对象，再经 setResponseFunctions（第八章）交给 HAL——回调接口本身就是按"电话域"拆分的双声道，这在 2021 年的 AIDL 化（radio 2.0，IRadioData/IRadioNetwork/…按域拆接口）里走到了明面。

7.5 交叉引用

- 第二章：socket→IRadio HAL 的通道演进（本章管"信封怎么写"，它管"信走哪条路"）。
- 第八章：RIL 实例的创建时机与 HAL 绑定的开机全链。
- 第九章：UNSOL_RESPONSE_NETWORK_STATE_CHANGED（1002，RILConstants.java:543）驱动 pollState 的完整闭环。
- 第十章：RIL_REQUEST_DIAL（10，RILConstants.java:361）在入队-出队-直方图上的完整过山车。

第八章 开机注册链：从 PhoneFactory 到 IRadio HAL 绑定

> **设计洞察 · 学习目标**
> 1. 复述 phone 进程启动时 makeDefaultPhone 的初始化顺序（socket 互斥→RIL 数组→UiccController→SubscriptionController→createPhone）；2. 说出 GsmCdmaPhone 构造函数组装的子系统及其依赖次序；3. 描述 RIL 与 IRadio HAL 的绑定过程（AIDL 优先、HIDL 版本瀑布兜底、setResponseFunctions）；4. 解释 HAL 死亡通知与代理重置的自愈机制

链路位置：PhoneFactory.makeDefaultPhones 是 telephony 王国的"开国仪式"。调用者是 com.android.phone 进程（packages/services/Telephony）的 PhoneGlobals（`PhoneGlobals.java:444` 调 `PhoneFactory.makeDefaultPhones(this)`）——AMS 拉起 phone 进程后，这里就是 Java 电话栈一切实例的起点。本章走通"进程启动→Phone 就绪→与 modem 打通"三段。

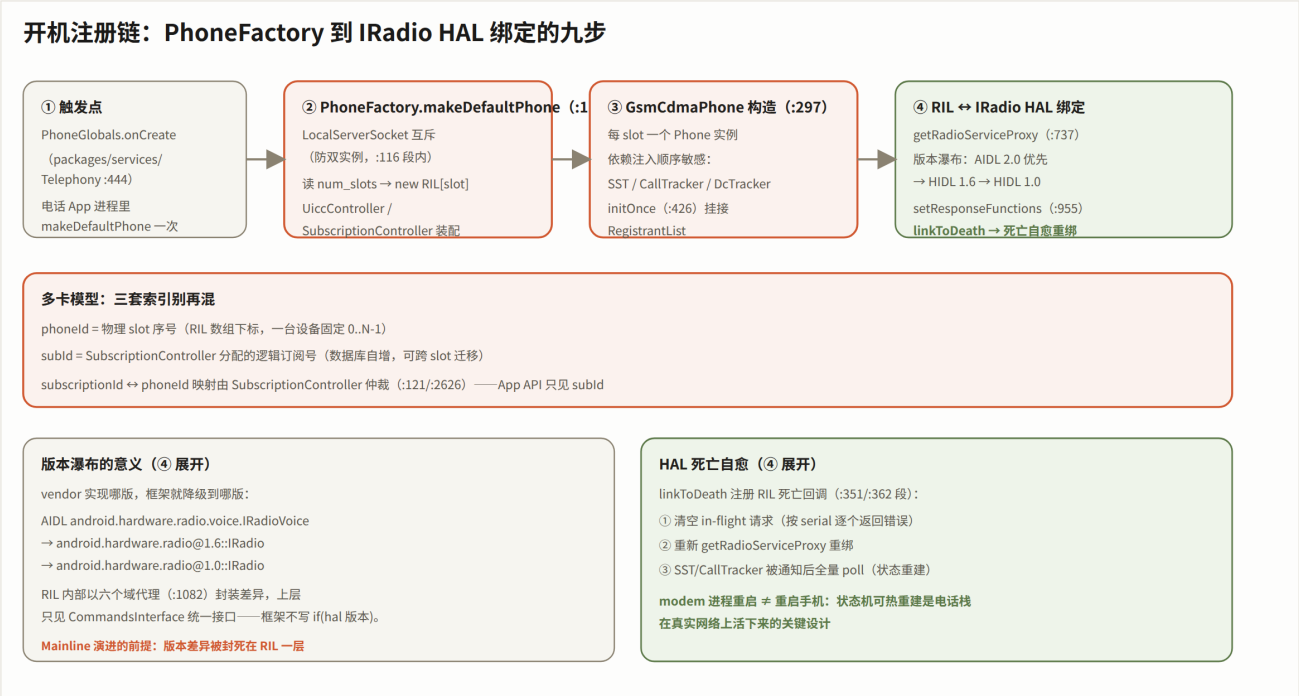


图 2 开机注册链九步与多卡三套索引：版本瀑布把 HAL 差异封死在 RIL 一层，死亡自愈支持 modem 热重建

8.1 makeDefaultPhone：九步开国

java frameworks/opt/telephony/…/PhoneFactory.java:116-165（节选+讲解注释）

```
@UnsupportedAppUsage
public static void makeDefaultPhone(Context context) {
synchronized (sLockProxyPhones) {
if (!sMadeDefaults) {
sContext = context;
// 讲解注释①：sMadeDefaults 布尔 + LocalServerSocket 双保险——
// 同进程防重复，跨进程防双开
int retryCount = 0;
for(;;) {
boolean hasException = false;
retryCount ++;
try {
// use UNIX domain socket to
// prevent subsequent initialization
new LocalServerSocket("com.android.internal.telephony");
} catch (java.io.IOException ex) {
hasException = true;
}
// 讲解注释②：Unix domain socket 抢占式互斥——第二个 phone
// 进程 bind 同名 socket 会失败，重试若干次后抛
// "PhoneFactory probably already running"
if ( !hasException ) { break; }
else if (retryCount > SOCKET_OPEN_MAX_RETRY) {
throw new RuntimeException("PhoneFactory probably already running");
} else { Thread.sleep(SOCKET_OPEN_RETRY_MILLIS); }
}
sPhoneNotifier = new DefaultPhoneNotifier(context);
// 讲解注释③：PhoneNotifier 是 Phone 向上（TelephonyRegistry/
```

```
// system_server）通知状态的出口——详见附录 A 的 registry
int numPhones = TelephonyManager.getDefault().getActiveModemCount();
// 讲解注释④：读 modem 的逻辑卡槽数——双卡机 numPhones=2
sPhones = new Phone[numPhones];
sCommandsInterfaces = new RIL[numPhones];
for (int i = 0; i < numPhones; i++) {
networkModes[i] = RILConstants.PREFERRED_NETWORK_MODE;
sCommandsInterfaces[i] = new RIL(context,
RadioAccessFamily.getRafFromNetworkType(networkModes[i]),
cdmaSubscription, i);
// 讲解注释⑤：每张逻辑卡一个 RIL 实例（phoneId=i）——
// RIL 构造函数见 :1024，第七章的管线从这里出生
}
sUiccController = UiccController.make(context);
// 讲解注释⑥：UiccController 管 SIM 卡（UICC）状态——
// 单例，所有 Phone 共享
TelephonyComponentFactory.getInstance().inject(SubscriptionController.
class.getName()).initSubscriptionController(context);
// 讲解注释⑦：SubscriptionController（ISub.Stub 的 Binder 服务）
// 在此注册——subId/phoneId 映射（多卡路由）开始可用
for (int i = 0; i < numPhones; i++) {
sPhones[i] = createPhone(context, i);
// 讲解注释⑧：逐卡创建 Phone 对象——见下
}
if (numPhones > 0) sPhone = sPhones[0];
// 讲解注释⑨：卡 1 兼任"default phone"——getPhone() 无参重载的返回值
...
}
}
}
```

createPhone 只有一件事——通过 TelephonyComponentFactory 反射点牌构造 GsmCdmaPhone（PhoneFactory.java:316）：

java frameworks/opt/telephony/…/PhoneFactory.java:316-328（真实代码+讲解注释）

```
private static Phone createPhone(Context context, int phoneId) {
int phoneType = TelephonyManager.getPhoneType(RILConstants.PREFERRED_NETWORK_MODE);
// 讲解注释①：PREFERRED_NETWORK_MODE 决定初始 phoneType；
// 注释原文 "We always use PHONE_TYPE_CDMA_LTE now"——CDMA 亦归入 LTE
if (phoneType == PHONE_TYPE_CDMA) phoneType = PHONE_TYPE_CDMA_LTE;
TelephonyComponentFactory injectedComponentFactory =
TelephonyComponentFactory.getInstance().inject(GsmCdmaPhone.class.getName());
// 讲解注释②：ComponentFactory 是 CTS/测试的注入缝——生产环境即 new
return injectedComponentFactory.makePhone(context,
sCommandsInterfaces[phoneId], sPhoneNotifier, phoneId, phoneType,
TelephonyComponentFactory.getInstance());
// 讲解注释③：CommandsInterface（即 RIL）作为构造参数注入——
// Phone 与 RIL 的绑定关系在此定型
}
```

8.2 GsmCdmaPhone 构造：子系统组装顺序

GsmCdmaPhone 构造函数（GsmCdmaPhone.java:297）是全书信息密度最高的 70 行——一部手机的全部子系统在此装配，且**次序即依赖**：

java frameworks/opt/telephony/…/GsmCdmaPhone.java:297-330（节选+讲解注释）

```
public GsmCdmaPhone(Context context, CommandsInterface ci, PhoneNotifier notifier,
boolean unitTestMode, int phoneId, int precisePhoneType,
TelephonyComponentFactory telephonyComponentFactory,
ImsManagerFactory imsManagerFactory) {
super(precisePhoneType == PhoneConstants.PHONE_TYPE_GSM ? "GSM" : "CDMA",
notifier, context, ci, unitTestMode, phoneId, telephonyComponentFactory);
// 讲解注释①：父类 Phone 记下名字/phoneId/mCi（CommandsInterface）
mPrecisePhoneType = precisePhoneType;
initOnce(ci);
// 讲解注释②（:426）：只做一次的事——mCT=GsmCdmaCallTracker（:429）、
// 注册 EVENT_RADIO_AVAILABLE/ON 等全套 RIL 监听
initRatSpecific(precisePhoneType);
mSST = mTelephonyComponentFactory.inject(ServiceStateTracker.class.getName())
.makeServiceStateTracker(this, this.mCi);
// 讲解注释③：SST 在 SSC 之后创建——注释原文 "SST/DSM depends on SSC"
if (!isUsingNewDataStack()) {
for (int transport : mAccessNetworksManager.getAvailableTransports()) {
DcTracker dcTracker = mTelephonyComponentFactory.inject(
DcTracker.class.getName()).makeDcTracker(this, transport);
mDcTrackers.put(transport, dcTracker);
}
// 讲解注释④：旧数据栈按 transport（WWAN/WLAN）各挂一个 DcTracker；
// 新栈（isUsingNewDataStack，Phone.java:4973）走 DataNetworkController
// ——注释原文预告 "This flag and the old data stack code will be
// deleted in Android 14"
}
mSST.registerForNetworkAttached(this, EVENT_REGISTERED_TO_NETWORK, null);
// 讲解注释⑤：注册"注网成功"事件——数据连接（第四章）等的就是它
CallManager.getInstance().registerPhone(this);
// 讲解注释⑥：把本 Phone 挂进 CallManager 总表——多 Phone 查询入口
}
```

设计洞察 · 逐段精读：为什么构造次序不能乱

SST 依赖 SignalStrengthController（读信号要 SSC 先就绪）、DcTracker 依赖 SST 与 DisplayInfoController（源注释写明 "DcTracker uses ServiceStateTracker and DisplayInfoController so needs to be created after"）、CarrierSignalAgent 依赖 CarrierActionAgent。Java 没有顶部排序的依赖图，telephony 用"构造函数书写顺序=初始化顺序"硬编码了这张图——改行序即改依赖，这是读懂 opt/telephony 任何大构造函数的钥匙。

8.3 与 IRadio HAL 绑定：AIDL 优先的版本瀑布

Phone/RIL 对象就绪后，真正的"打通 modem"发生在第一次实际请求时——getRadioServiceProxy（RIL.java:737）按需服务化绑定，Android 13 的策略是 **AIDL（radio 2.0）优先，HIDL 1.6→1.0 瀑布兜底**：

java frameworks/opt/telephony/…/RIL.java:737-780、955-974（绑定+讲解注释）

```
public synchronized RadioServiceProxy getRadioServiceProxy(int service, Message result) {
...
RadioServiceProxy serviceProxy = mServiceProxies.get(service);
if (!serviceProxy.isEmpty()) {
return serviceProxy;
// 讲解注释①：已绑定直接复用——六个域代理（data/messaging/
// modem/network/sim/voice）在构造函数 :1082 起预置空壳
}
switch (service) {
case DATA_SERVICE:
binder = ServiceManager.waitForDeclaredService(
android.hardware.radio.data.IRadioData.DESCRIPTOR + "/"
+ HIDL_SERVICE_NAME[mPhoneId]);
// 讲解注释②：ServiceManager 按描述符+实例名找 HAL——
// 实例名 slot0/slot1 对应双卡（HIDL_SERVICE_NAME[phoneId]）
if (binder != null) {
mRadioVersion = RADIO_HAL_VERSION_2_0;
((RadioDataProxy) serviceProxy).setAidl(mRadioVersion,
android.hardware.radio.data.IRadioData.Stub.asInterface(binder));
}
// 讲解注释③：AIDL 版直接 asInterface——Binder 标准姿势
break;
...
}
if (serviceProxy.isEmpty() && mRadioVersion.less(RADIO_HAL_VERSION_2_0)) {
try {
mRadioVersion = RADIO_HAL_VERSION_1_6;
serviceProxy.setHidl(mRadioVersion,
android.hardware.radio.V1_6.IRadio.getService(
HIDL_SERVICE_NAME[mPhoneId], true));
} catch (NoSuchElementException e) { }
// 讲解注释④：HIDL 瀑布——1.6 失败再试 1.5/1.4/…/1.0（:860-905
// 六连 try），厂商 HAL 装多老都能接住
}
if (!serviceProxy.isEmpty()) {
if (serviceProxy.isAidl()) {
// （节选自 :955 起，VOICE_SERVICE 分支）
mDeathRecipients.get(service).linkToDeath(
((RadioVoiceProxy) serviceProxy).getAidl().asBinder());
((RadioVoiceProxy) serviceProxy).getAidl().setResponseFunctions(
mVoiceResponse, mVoiceIndication);
// 讲解注释⑤：绑定即互相交换回调——setResponseFunctions 把
// 第七章的 Response/Indication 对象交给 HAL 侧，
// modem 从此能主动敲门
} else {
serviceProxy.getHidl().linkToDeath(mRadioProxyDeathRecipient,
mServiceCookies.get(service).incrementAndGet());
serviceProxy.getHidl().setResponseFunctions(mRadioResponse, mRadioIndication);
// 讲解注释⑥：HIDL 分支同理，cookie 递增用于甄别"死亡通知
// 是否属于当前绑定"
}
}
}
```

hal 层的接口长这样（`hardware/interfaces/radio/aidl/android/hardware/radio/voice/IRadioVoice.aidl`）：`oneway void setResponseFunctions(in IRadioVoiceResponse, in IRadioVoiceIndication)`——oneway 双向：请求是 oneway 调 HAL，上报是 oneway 调回 Java。**这正是第二章"socket 模型"的现代形态：socket 换成了 Binder 直通，但"命令号+serial 请求、无序号上报"的 RIL 语义原样保留**（语义规范仍在 RILConstants.java 与 hardware/ril/libril 的参考实现里）。

8.4 死亡与自愈

HAL 进程崩溃不会让电话栈瘫痪：RilHandler 的 EVENT_RADIO_PROXY_DEAD/EVENT_AIDL_PROXY_DEAD（RIL.java:351/362）收到死亡通知后比对 cookie，确认是当前绑定则 `resetProxyAndRequestList(service)`——清空代理、给账本上所有在途请求回 RADIO_NOT_AVAILABLE 错误，等下次请求触发重新绑定（8.3 的瀑布再来一遍）。这就是排障手册里"modem 重启后电话栈自愈"的源码依据。

> **设计洞察 · 设计洞察：为什么绑定放在"第一次请求"而非构造时**
> RIL 构造发生在 phone 进程启动早期，而厂商 HAL 服务可能还没注册进 ServiceManager（modem 自启较慢）。getRadioServiceProxy 的"按需+同步等待（waitForDeclaredService）"把时序竞争变成了首次请求的毫秒级等待；配合死亡重绑，整套机制成为一个可无限重试的幂等操作——Telephony 用最朴素的方式对抗 modem 世界的不确定性。

8.5 交叉引用

- 第三章：GsmCdmaPhone 状态清单的静态视图（本章是它的构造时刻）。
- 第七章：RIL 数组创建（注释⑤）后即有管线；setResponseFunctions 交付的正是那 7 组 Response/Indication。
- 第九章：SST（构造注释③）创建后立刻随 EVENT_RADIO_ON 启动首次 pollState——开机注册链的下一棒。
- 附录 B：PhoneFactory 问答条目。

第九章 网络注册状态机：ServiceStateTracker 的双引擎

> **设计洞察 · 学习目标**
> 1. 说出注册状态的两条触发路径（unsol 事件与主动 poll）；2. 描述 pollStateInternal 的四联请求与 mPollingContext 倒计数聚合；3. 走读 handlePollStateResultMessage 如何把 NetworkRegistrationInfo 写入 mNewSS；4. 解释 pollStateDone 的双缓冲提交与 stale 检查

链路位置：状态栏左上角那格信号，背后是 ServiceStateTracker（`ServiceStateTracker.java`，6062 行，本章简称 SST）的状态机。链路：modem 注册状态变化 → `RIL_UNSOL_RESPONSE_NETWORK_STATE_CHANGED`（1002）→ NetworkIndication.networkStateChanged（:238，第七章已读）→ mNetworkStateRegistrants → SST。这是 Android 13 最重要的"设备在网状态"源头，telephony.registry 的 mServiceState（附录 A）就是它发布的。

9.1 双引擎：事件驱动与轮询核对

SST 收到 unsol 后并不直接信任上报内容，而是**发起一轮主动 poll**——事件是"变更通知"，poll 结果才是"事实核对"：

java frameworks/opt/telephony/…/ServiceStateTracker.java:672-676、1335-1341（订阅与入口+讲解注释）

```
// ===== 构造函数里的订阅（SST :672）=====
for (int transportType : mAccessNetworksManager.getAvailableTransports()) {
mRegStateManagers.append(transportType, new NetworkRegistrationManager(
transportType, phone));
mRegStateManagers.get(transportType).registerForNetworkRegistrationInfoChanged(
this, EVENT_NETWORK_STATE_CHANGED, null);
}
// 讲解注释①：按 transport（WWAN 蜂窝/WLAN_IWLAN）各建一个
// NetworkRegistrationManager 订阅注册信息变化——Android 13 把
// 旧式 CREG/CGREG 细分事件收拢为统一的 NetworkRegistrationInfo
// ===== handleMessage 入口（SST :1335）=====
case EVENT_NETWORK_STATE_CHANGED:
pollStateInternal(true);
// 讲解注释②：unsol 到达不直接改状态——立刻发起一轮 modemTriggered
// 的 poll（9.2），事件只是"该去查了"的铃声
break;
```

pollState 本体只有一行（`ServiceStateTracker.java:3346`：`sendEmptyMessage(EVENT_POLL_STATE_REQUEST)`），真正干活的是 pollStateInternal（:3353）。

9.2 pollStateInternal：四联请求与倒计数

java frameworks/opt/telephony/…/ServiceStateTracker.java:3353-3417（节选+讲解注释）

```
private void pollStateInternal(boolean modemTriggered) {
mPollingContext = new int[1];
// 讲解注释①：轮询上下文是个长度为 1 的 int 数组——作为"共享计数器"
// 随 message 往返，四个响应都要靠它认亲
switch (mCi.getRadioState()) {
case TelephonyManager.RADIO_POWER_UNAVAILABLE:
...
mNewSS.setOutOfService(mAccessNetworksManager.isInLegacyMode(), false);
// 讲解注释②：radio 不可用——但 IWLAN 注册态要保留（先取出，
// outOfService 后再 addNetworkRegistrationInfo 加回去）
...
pollStateDone();
break;
case TelephonyManager.RADIO_POWER_OFF:
```

```
...
if (mDeviceShuttingDown ||
(!modemTriggered && ServiceState.RIL_RADIO_TECHNOLOGY_IWLAN
!= mSS.getRilDataRadioTechnology())) {
pollStateDone();
break;
}
// 讲解注释③：关机/非 modem 触发则免轮询直接出服务态
default:
// Issue all poll-related commands at once then count down the responses,
// which are allowed to arrive out-of-order
mPollingContext[0]++;
mCi.getOperator(obtainMessage(EVENT_POLL_STATE_OPERATOR, mPollingContext));
// 讲解注释④：第一发——查运营商名（AT+COPS）
mPollingContext[0]++;
mRegStateManagers.get(AccessNetworkConstants.TRANSPORT_TYPE_WWAN)
.requestNetworkRegistrationInfo(NetworkRegistrationInfo.DOMAIN_PS,
obtainMessage(EVENT_POLL_STATE_PS_CELLULAR_REGISTRATION,
mPollingContext));
// 讲解注释⑤：第二发——PS 域（数据）蜂窝注册态（AT+CGREG）
mPollingContext[0]++;
mRegStateManagers.get(AccessNetworkConstants.TRANSPORT_TYPE_WWAN)
.requestNetworkRegistrationInfo(NetworkRegistrationInfo.DOMAIN_CS,
obtainMessage(EVENT_POLL_STATE_CS_CELLULAR_REGISTRATION,
mPollingContext));
// 讲解注释⑥：第三发——CS 域（语音）注册态（AT+CREG）
if (mRegStateManagers.get(AccessNetworkConstants.TRANSPORT_TYPE_WLAN) != null) {
mPollingContext[0]++;
mRegStateManagers.get(AccessNetworkConstants.TRANSPORT_TYPE_WLAN)
.requestNetworkRegistrationInfo(NetworkRegistrationInfo.DOMAIN_PS,
obtainMessage(EVENT_POLL_STATE_PS_IWLAN_REGISTRATION,
mPollingContext));
}
// 讲解注释⑦：第四发（可选）——IWLAN 注册态
// 计数器=在途请求数，每个响应回来减一，归零即提交（见 9.3）
}
}
```

四发请求**一次性并发**，注释原文明说允许乱序返回（allowed to arrive out-of-order）——聚合靠 mPollingContext。

9.3 handlePollStateResult：认亲、排错、归零提交

java frameworks/opt/telephony/…/ServiceStateTracker.java:1937-1982（节选+讲解注释）

```
protected void handlePollStateResult(int what, AsyncResult ar) {
// Ignore stale requests from last poll
if (ar.userObj != mPollingContext) return;
// 讲解注释①：stale 检查——响应携带的计数器引用若不是当前这轮的
// mPollingContext，说明是上一轮的迟到响应，直接丢弃
if (ar.exception != null) {
...
if (mCi.getRadioState() != TelephonyManager.RADIO_POWER_ON) {
log("handlePollStateResult: Invalid response due to radio off or "
+ "unavailable. Set ServiceState to out of service.");
```

```
pollStateInternal(false);
return;
// 讲解注释②：radio 状态中途翻转——整轮作废重开
}
if (err == CommandException.Error.RADIO_NOT_AVAILABLE) {
loge("handlePollStateResult: RIL returned RADIO_NOT_AVAILABLE when "
+ "radio is on.");
cancelPollState();
return;
// 讲解注释③：HAL 不可用（第八章的死亡重置窗口）——
// 取消本轮，等 radio 状态事件重新驱动
}
} else try {
handlePollStateResultMessage(what, ar);
// 讲解注释④：正常响应——按 what 分派到四个解析分支（9.4）
} catch (RuntimeException ex) {
loge("Exception while polling service state. Probably malformed RIL "
+ "response." + ex);
// 讲解注释⑤：modem 报文畸形也不许崩——吃掉异常继续计数
}
mPollingContext[0]--;
if (mPollingContext[0] == 0) {
mNewSS.setEmergencyOnly(mEmergencyOnly);
combinePsRegistrationStates(mNewSS);
updateOperatorNameForServiceState(mNewSS);
// 讲解注释⑥：四发全归零——把散落的解析结果合并进 mNewSS，
// 随后走 roaming 判定、pollStateDone 提交（:3460）
}
}
```

9.4 handlePollStateResultMessage：CS 分支精读

java frameworks/opt/telephony/…/ServiceStateTracker.java:2214-2245（节选+讲解注释）

```
protected void handlePollStateResultMessage(int what, AsyncResult ar) {
switch (what) {
case EVENT_POLL_STATE_CS_CELLULAR_REGISTRATION: {
NetworkRegistrationInfo networkRegState = (NetworkRegistrationInfo) ar.result;
// 讲解注释①：RIL 层已把 AT+CREG 报文解析成
// NetworkRegistrationInfo 对象——SST 不再见原始字符串
int registrationState = networkRegState.getRegistrationState();
int newVoiceRat = ServiceState.networkTypeToRilRadioTechnology(
networkRegState.getAccessNetworkTechnology());
mNewSS.setVoiceRegState(regCodeToServiceState(registrationState));
// 讲解注释②：注册码→服务态（regCodeToServiceState，:4203）——
// HOME/ROAMING=IN_SERVICE，其余=OUT_OF_SERVICE；
// "仅紧急呼叫"由 isEmergencyEnabled 单独记入 mEmergencyOnly
mNewSS.addNetworkRegistrationInfo(networkRegState);
setPhyCellInfoFromCellIdentity(mNewSS, networkRegState.getCellIdentity());
// 讲解注释③：小区身份（MCC/MNC/LAC/CID）写入 ServiceState
int reasonForDenial = networkRegState.getRejectCause();
// 讲解注释④：拒绝原因（registrationState=3 时）——排障时
// "注网被拒"的关键字段
mEmergencyOnly = (mCSEmergencyOnly || mPSEmergencyOnly);
...
```

```
if (DBG) {
log("handlePollStateResultMessage: CS cellular. " + networkRegState);
}
break;
}
...
}
}
```

9.5 pollStateDone：双缓冲提交

mNewSS（候选态）与 mSS（当前态）是双缓冲：四发解析全部写入 mNewSS，pollStateDone（`ServiceStateTracker.java:3460`）比对二者，有变化才原子提交 mSS=...、广播 ServiceState 变化（DefaultPhoneNotifier → TelephonyRegistry → 全系统 telephony.registry 与状态栏），并联动 SignalStrengthController/DisplayInfoController 等兄弟控制器。

> **设计洞察 · 设计洞察：为什么"通知+轮询"双引擎而不是纯事件**
> modem 上报可能丢、可能迟、可能在 AP 深睡时被吞——纯事件驱动的注册态会慢慢漂移。SST 用 unsol 触发 poll、用 poll 结果说话，把最终一致性锚定在"每次变化后的一轮完整四联查询"上。
> mPollingContext 数组引用做 stale 甄别同样是教科书手法：不用时间戳、不用序号，一个引用相等判断就筛掉了上轮迟到响应——代价只是每轮 new int[1]。

9.6 交叉引用

- 第七章：UNSOL 从 HAL Indication 到 RegistrantList 的前半程（本章是后半程）。
- 第八章：EVENT_RADIO_ON 后 SST 的首次 pollState 即开机注网的起点；RADIO_NOT_AVAILABLE 分支与 HAL 死亡重置衔接。
- 第四章：注册成功（registerForNetworkAttached，GsmCdmaPhone.java:366）是数据连接（DcTracker）开闸的闸门。
- 附录 A：dumpsys telephony.registry 看到的就是 mSS 提交后的广播。

第十章 呼叫建立链：从 Telecom 到 CLCC 轮询

> **设计洞察 · 学习目标**
> 1. 走通 Dialer 到 modem 的后四跳（TelephonyConnectionService→GsmCdmaPhone→GsmCdmaCallTracker→RIL）；2. 解读 dialGsm 的 checkForDialIssues/fakeHoldForegroundBeforeDial/pendingMO 三板斧；3. 说出 RIL_REQUEST_DIAL（10）与 RIL_UNSOL_RESPONSE_CALL_STATE_CHANGED（1001）的配合关系；4. 解释 handlePollCalls 如何用 CLCC 列表比对推进 Call 状态

链路位置：第三章给了七跳全景（Dialer→Telecom→ConnectionService→Phone→RIL→rild/modem→回）。本章逐行走读 phone 进程内的后四跳。入口在 packages/services/Telephony 的 TelephonyConnectionService（`TelephonyConnectionService.java:1733` 调 `phone.dial(...)`，

Telecom 的 ConnectionService 回调点）→ GsmCdmaPhone.dial（GsmCdmaPhone.java:1405，做紧急号改写与 IMS 分流）→ mCT.dial（GsmCdmaCallTracker.java:554）→ dialGsm（:281）。

10.1 dialGsm：拨号请求的装配车间

java frameworks/opt/telephony/…/GsmCdmaCallTracker.java:281-368（节选+讲解注释）

```
public synchronized Connection dialGsm(String dialString, DialArgs dialArgs)
throws CallStateException {
int clirMode = dialArgs.clirMode;
...
boolean isEmergencyCall = dialArgs.isEmergency;
if (isEmergencyCall) {
clirMode = CommandsInterface.CLIR_SUPPRESSION;
// 讲解注释①：紧急呼叫必须上报本机号码（压制主叫隐藏）——
// 救援中心要能回拨
}
// note that this triggers call state changed notif
clearDisconnected();
// 讲解注释②：先清场——把已挂断未清理的 Connection 回收，
// 腾出 mConnections[6] 数组槽位（GSM 一路通话占一槽）
checkForDialIssues(isEmergencyCall);
// 讲解注释③：前置检查（:canDial）——radio off/通话中限制/EMC 优先，
// 不通过直接抛 CallStateException 给 Telecom
String origNumber = dialString;
dialString = convertNumberIfNecessary(mPhone, dialString);
// 讲解注释④：运营商改写规则（carrier config 的号码翻译）在此生效
if (mForegroundCall.getState() == GsmCdmaCall.State.ACTIVE) {
switchWaitingOrHoldingAndActive();
// 讲解注释⑤：前台通话占线——先切换（AT+CHLD），把老话务压后台
try {
Thread.sleep(500);
} catch (InterruptedException e) {
// do nothing
}
// 讲解注释⑥：代码注释自称 "a hack to delay DIAL"——等 SWITCH
// 在 modem 侧落地，500ms 是经验值而非事件同步（见本章精读盒）
fakeHoldForegroundBeforeDial();
// 讲解注释⑦：本地伪造"前台已清空"——下一个 poll 不会把新号
// 误判成挂断
}
if (mForegroundCall.getState() != GsmCdmaCall.State.IDLE) {
throw new CallStateException("cannot dial in current state");
// 讲解注释⑧：兜底——仍不空闲就拒绝，宁可失败不可错乱
}
mPendingMO = new GsmCdmaConnection(mPhone, dialString, this, mForegroundCall,
dialArgs);
// 讲解注释⑨：本端发起的呼叫先当"未证实"的 pendingMO 挂起——
// 它还不存在于 modem 的 CLCC 列表里
...
mMetrics.writeRilDial(mPhone.getPhoneId(), mPendingMO, clirMode, uusInfo);
if ( mPendingMO.getAddress() == null || mPendingMO.getAddress().length() == 0
|| mPendingMO.getAddress().indexOf(PhoneNumberUtils.WILD) >= 0) {
mPendingMO.mCause = DisconnectCause.INVALID_NUMBER;
```

```
pollCallsWhenSafe();
// 讲解注释⑩：空号/通配符——不发 RIL，靠 poll 把它标记为拨号失败
} else {
setMute(false);
mCi.dial(mPendingMO.getAddress(), mPendingMO.isEmergencyCall(),
mPendingMO.getEmergencyNumberInfo(),
mPendingMO.hasKnownUserIntentEmergency(),
clirMode, uusInfo, obtainCompleteMessage());
// 讲解注释⑪：RIL_REQUEST_DIAL（10）在此出膛——obtainCompleteMessage
// 带回一个 EVENT_OPERATION_COMPLETE 回调
}
}
```

RIL 侧（RIL.java:1471 dial）把参数打包成 AIDL 的 Dial 结构交给 RadioVoiceProxy（RadioVoiceProxy.java:122），最终 IRadioVoice.dial(serial, dialInfo)（oneway Binder 调用）抵达厂商 HAL——serial 配对的故事见第七章。

10.2 UNSOL 驱动：从 modem 敲门到轮询列表

modem 发起建立后会上报 RIL_UNSOL_RESPONSE_CALL_STATE_CHANGED（1001）：

java frameworks/opt/telephony/…/VoiceIndication.java:92-96（真实代码+讲解注释）

```
public void callStateChanged(int indicationType) {
mRil.processIndication(RIL.VOICE_SERVICE, indicationType);
if (RIL.RILJ_LOGD) mRil.unsljLog(RIL_UNSOL_RESPONSE_CALL_STATE_CHANGED);
// 讲解注释①：logcat -b radio 的 "UNSOL_CALL_STATE_CHANGED" 出处
mRil.mCallStateRegistrants.notifyRegistrants();
// 讲解注释②：广播——GsmCdmaCallTracker 在构造时注册
//  （GsmCdmaCallTracker.java:159 registerForCallStateChanged）
}
```

GsmCdmaCallTracker 收到后同样"事件只当闹钟"——进入 poll 流程：

java frameworks/opt/telephony/…/GsmCdmaCallTracker.java:1639-1643 与 831-895（节选+讲解注释）

```
case EVENT_REPOLL_AFTER_DELAY:
case EVENT_CALL_STATE_CHANGE:
pollCallsWhenSafe();
// 讲解注释①：unsol/重试定时器殊途同归——发起 RIL_REQUEST_GET_CURRENT_CALLS
...
protected synchronized void handlePollCalls(AsyncResult ar) {
List polledCalls;
if (ar.exception == null) {
polledCalls = (List)ar.result;
// 讲解注释②：ar.result 是 RIL 解析好的 DriverCall 列表
//  （CLCC 报文：索引/方向/状态/号码）
} else if (isCommandExceptionRadioNotAvailable(ar.exception)) {
polledCalls = new ArrayList();
// 讲解注释③：radio 没了——用空列表触发"全部挂断"清理
} else {
pollCallsAfterDelay();
return;
}
Connection newRinging = null; //or waiting
```

```
boolean hasNonHangupStateChanged = false;
for (int i = 0, curDC = 0, dcSize = polledCalls.size()
; i < mConnections.length; i++) {
GsmCdmaConnection conn = mConnections[i];
DriverCall dc = null;
if (curDC < dcSize) {
dc = (DriverCall) polledCalls.get(curDC);
if (dc.index == i+1) { curDC++; } else { dc = null; }
// 讲解注释④：本端槽位数组与网络列表按 index 对齐——
// 两边各自的"账"逐槽比对
}
if (conn == null && dc != null) {
// Connection appeared in CLCC response that we don't know about
if (mPendingMO != null && mPendingMO.compareTo(dc)) {
mConnections[i] = mPendingMO;
mPendingMO.mIndex = i;
mPendingMO.update(dc);
mPendingMO = null;
// 讲解注释⑤：10.1 的 pendingMO 在 CLCC 里现身了——
// "未证实"转正入册；compareTo 按号码匹配
} else if (dc.isVoice) {
newRinging = checkMtFindNewRinging(dc, i);
// 讲解注释⑥：陌生来话——走被叫（MT）路径，触发响铃
}
} else if (conn != null && dc == null) {
...
}
}
...
updatePhoneState();
// 讲解注释⑦：汇总出 PhoneConstants.State（IDLE/RINGING/OFFHOOK）——
// TelephonyManager.getCallState 与 InCallUI 由此感知
mPhone.notifyNewRingingConnection(newRinging);
...
}
```

10.3 Call 状态推进全景

一条 MO 呼叫的完整状态推进：DIALING（pendingMO 本地态）→ OPERATION_COMPLETE 后 CLCC 里出现 DIALING → ALERTING（对端振铃）→ ACTIVE（接通），每一步都是"一次 UNSOL 闹钟 + 一轮 CLCC 轮询"的循环；挂断同样由 CLCC 消失驱动。Call/Connection 的对外状态（GsmCdmaCall.State.DIALING/ALERTING/ACTIVE…）只是这套比对的投影。

> **设计洞察 · 逐段精读：Thread.sleep(500) 的设计张力**
>
> dialGsm 里那段注释自称 hack 的 500ms 睡眠，暴露了 telephony 的一个深层约束：多路通话的 SWITCH（AT+CHLD）与 DIAL 在 modem 侧有隐式时序依赖，而 RIL 并没有提供"SWITCH 完成事件"（EVENT_SWITCH_RESULT 到达≠modem 侧执行完毕）。事件驱动在这类网络黑盒面前失效，工程上只能用时间窗硬扛——这也是第九章"poll 核对"哲学的代价注脚：modem 语义不完整处，轮询与魔法数补位。

> **设计洞察 · 设计洞察：CLCC 比对是 AP 与 modem 的对账机制**
> CLCC（当前呼叫列表）轮询本质是"以 modem 的账为准、AP 逐槽对账"：新出现的槽=新呼叫（MO 转正或 MT 响铃），消失的槽=挂断，状态变化=DIALING→ACTIVE。事件（UNSOL_CALL_STATE_CHANGED）只保证"该对账了"，对账结果才对外发布。这套机制让 GsmCdmaCallTracker 无需实现 modem 侧每种边缘时序——健壮性来自最后一致性，而非实时同步。

10.4 交叉引用

- 第三章：七跳全景（本章走读其第 4-7 跳的 phone 进程内段）。
- 第七章：mCi.dial 的入队/配对/直方图即管线一次完整过山车；1001 与 1002 同属 Indication 双声道。
- 第五章：GsmCdmaPhone.dial（:1405）里的 IMS 分流——VoLTE 可用时 dial 走 ImsPhone，本章的 CS 路径是其回退。
- 附录 A：dumpsys telecom 的 Call 列表与 telephony.registry 的 mCallState 即本链对外投影。

教科书习题集（分级）

第一级：简答题（检验概念理解）

- **1. Telephony 的框架源码为什么住在 frameworks/opt/telephony 而不是 frameworks/base？**

参考答案要点：历史起源独立；telephony 每版大改而 core 稳定，演进节奏不同；独立目录为 Mainline/APEX 化铺路（第一章）。

- **2. RIL 与 modem 的请求-应答靠什么配对？**

参考答案要点：RILRequest 的 mSerial 全局递增（RILRequest.java:100）；发送前入 mRequestList 账本（RIL.java:1163），最终响应 findAndRemoveRequestFromList 按 serial 摘除（:5183）。

- **3. RIL_REQUEST_DIAL 与 RIL_UNSOL_RESPONSE_CALL_STATE_CHANGED 是什么？值各是多少？**

参考答案要点：请求命令号 10 与上报号 1001（RILConstants.java:361/542）；前者是 AP 主动建立呼叫，后者是 modem 通知呼叫列表变更（第十章）。

- **4. GsmCdmaCallTracker 收到 UNSOL_CALL_STATE_CHANGED 后为什么还要 poll CLCC？**

参考答案要点：事件只是"变更闹钟"，不带列表内容；以 modem 的 CLCC 列表为准逐槽对账（handlePollCalls，:831），保证最终一致。

- **5. ServiceStateTracker 一次 pollState 会并发发几个请求？如何聚合？**

参考答案要点：四发——运营商名/PS 蜂窝注册/CS 蜂窝注册/IWLAN 注册；mPollingContext 计数器随 message 往返，每个响应减一，归零触发提交（第九章）。

- **6. mNewSS 与 mSS 双缓冲的作用是什么？**

参考答案要点：候选态与当前态分离；pollStateDone（:3460）比对后有变化才提交并广播——杜绝半成品状态外泄。

- **7. makeDefaultPhone 里 new LocalServerSocket("com.android.internal.telephony") 的作用？**

参考答案要点：跨进程单例互斥——第二个 phone 进程 bind 同名 socket 失败，重试超限抛 "PhoneFactory probably already running"（PhoneFactory.java:131 起）。

- **8. setResponseFunctions 交换的是什么？**

参考答案要点：RIL 把 7 组 Response/Indication 回调对象交给 HAL（AIDL 按域拆接口，RIL.java:955 起）；HAL 由此能反向回调 Java——绑定即"互留电话"。

- **9. subId 与 phoneId 有何区别？SubscriptionController 如何映射？**

参考答案要点：subId 是订阅记录主键（telephony.db），phoneId 是逻辑卡槽；映射表 sSlotIndexToSubIds，getSubId（:2626）/getPhoneId（:2674）双向查（多卡路由）。

- **10. IMS 与 CS 域打电话的本质区别？**

参考答案要点：IMS 走 PS 域 IP 包（SIP 信令+RTP 语音），CS 是电路交换专用信道；VoLTE 开通=IMS 注册成功（第五章）。

第二级：分析题（检验机制理解）

- **11. 追踪一次 pollState 的完整往返，标注每次跨进程/跨 HAL 跳。**

参考答案要点：SST pollStateInternal→CommandsInterface→RadioServiceProxy→AIDL Binder（oneway）→vendor HAL→modem→HAL Indication 回调 →RadioNetworkProxy→NetworkIndication→RegistrantList→SST handleMessage——共四次跨边界（Java/Binder/HAL/modem）。

- **12. WAKE_LOCK_TIMEOUT 后迟到的响应还能配对吗？为什么？**

参考答案要点：能——超时只清 WakeLock 且注释明确 Keep mRequestList（RIL.java:295 起）；迟来响应仍走 processResponse 按 serial 摘除（第七章）。

- **13. vendor HAL 进程崩溃后电话栈如何恢复？**

参考答案要点：linkToDeath 触发 EVENT_AIDL_PROXY_DEAD（RIL.java:362）→比对 cookie→resetProxyAndRequestList：在途请求回 RADIO_NOT_AVAILABLE、代理清空→下次请求触发 8.3 版本瀑布重绑（第八章自愈）。

- **14. 若 handlePollStateResult 不做 stale 检查会怎样？**

参考答案要点：上一轮迟到响应会写入本轮 mPollingContext 计数与 mNewSS——可能提前"归零"提交半套结果，注册态闪烁或错乱（ServiceStateTracker.java:1940）。

- **15. dialGsm 里 Thread.sleep(500) 想解决什么？删掉会怎样？**

参考答案要点：等 SWITCH（AT+CHLD）在 modem 侧落地后再 DIAL（三方通话时序）；删掉可能 DIAL 先到被拒或老话务未被压后台——EVENT_SWITCH_RESULT 不等于 modem 执行完成，RIL 缺该语义只能时间窗补位（第十章精读盒）。

- **16. 双卡设备两条拨号请求为什么互不污染？**

参考答案要点：每 phoneId 一个 RIL 实例，各自独立 mRequestList/serial 空间；HAL 侧 slot0/slot1 双实例（PhoneFactory.java:158 起的数组构造）。

- **17. MO 呼叫从拨出到接通，Call 状态经历哪些变化？每步由什么驱动？**

参考答案要点：DIALING（pendingMO 本地）→CLCC 出现 DIALING→ALERTING→ACTIVE；每步都是"UNSOL 1001 闹钟+一轮 GET_CURRENT_CALLS 对账"循环（第十章）。

第三级：设计题

- **18. 设计一套 RIL 请求延迟监控，基于现有管线该在哪三个点插桩？**

参考答案要点：obtainRequest 已记 mStartTimeMs（起点）；processResponseInternal 已有 addToRilHistogram（5 桶终点）；WAKE_LOCK_TIMEOUT 处上报超时事件（RIL.java:292）——补一个按 request 号的 P50/P99 导出即可，无需改协议。

- **19. 假设删掉全部 UNSOL 只留轮询，系统行为如何变化？请权衡。**

参考答案要点：功能仍可用（定时 pollState/pollCalls 兜底），但来话/注网时延劣化一个轮询周期、耗电上升；结论——unsol 是性能与时延优化而非正确性必需；反向（删 poll 只留 unsol）则不成立，事件丢失即永久漂移。

- **20. 为"双卡双 modem 各自可能重启"设计故障隔离，要考虑哪些点？**

参考答案要点：代理/账本/cookie 按 phoneId 与 service 隔离（resetProxyAndRequestList 只清出事域）；死亡通知只重绑本实例；SubscriptionController 降级路由（mDefaultPhoneId 兜底，：2674）；避免一张卡的 RADIO_NOT_AVAILABLE 风暴触发另一卡重试退避连锁。

术语表

术语	解释
APN	Access Point Name，数据连接接入点配置（类型 default/mms/ims/supl 等，第四章）
CLCC	Current List of Calls，modem 的当前呼叫列表（AT+CLCC），CallTracker 对账的事实源（第十章）

术语	解释
DataConnection	数据连接状态机（dataconnection/DataConnection.java:146），一次 PDN 会话的生命周期（第四章）
DcTracker	旧数据栈的 APN 调度器（dataconnection/DcTracker.java:3110 onDataSetupComplete），Android 14 将被新栈取代（第八章）
DriverCall	RIL 解析 CLCC 报文得到的一路呼叫快照，handlePollCalls 的比对单元（第十章）
GsmCdmaCallTracker	呼叫跟踪器（GsmCdmaCallTracker.java:281 dialGsm），管理 mConnections 槽位与 Call 状态推进（第三/十章）
GsmCdmaPhone	2G/3G/4G 合一电话状态机（GsmCdmaPhone.java:141），一部手机状态的封装（第三章）
HAL	Hardware Abstraction Layer，此处指 vendor 实现的 IRadio 服务（android.hardware.radio.*，第八章）
IMS	IP Multimedia Subsystem，SIP 信令+RTP 语音的 IP 电话体系（第五章）
ImsPhone	挂在 GsmCdmaPhone 旁的 IMS Phone 实现，VoLTE 通话走它（第五章）
IRadio	modem HAL 接口族，Android 13 含 AIDL 2.0 与 HIDL 1.0-1.6 两套（第八章）
PhoneFactory	Phone 实例工厂（PhoneFactory.java:116 makeDefaultPhone），开机创建 RIL 数组与 Phone 数组（第八章）
phoneId	逻辑卡槽编号（双卡为 0/1），RIL 实例与 HAL 实例名的索引（多卡路由）
pollState	SST 主动轮询注册态（ServiceStateTracker.java:3346），四联请求倒计数聚合（第九章）
RadioServiceProxy	RIL 持有的 HAL 代理基类，按域拆出 Data/Sim/Network/Voice 等 6 个子代理（第七/八章）
RegistrantList	观察者名单，unsol 消息广播给所有 registerForXxx 的状态机（第七章）
RIL	Radio Interface Layer（RIL.java:108），Java 框架与 modem 的唯一桥（第二章）
RILConstants	RIL 请求/上报命令号总表（RILConstants.java:361 DIAL=10 等，第七章）
RILRequest	一次 modem 请求的 Java 对象（RILRequest.java:36），serial 配对主键+对象池复用（第七章）
SerialNumber	RILRequest 的全局递增序号，请求与应答的配对主键（第七章）

术语	解释
ServiceState	服务态快照（语音/数据注册态、漫游、运营商名），mNewSS/mSS 双缓冲（第九章）
ServiceStateTracker	注册态跟踪器（ServiceStateTracker.java:1937 handlePollStateResult），unsol+poll 双引擎（第九章）
setResponseFunctions	HAL 绑定握手：RIL 把 Response/Indication 回调交给 vendor 侧（RIL.java:955 起，第八章）
subId	订阅记录主键（telephony.db siminfo 表），多卡路由的"身份"（SubscriptionController）
SubscriptionController	subId/phoneId 映射的 Binder 服务（SubscriptionController.java:121 extends ISub.Stub，多卡路由）
TelephonyProvider	telephony.db 数据库（APN/siminfo），PhoneFactory 启动链一环（第一章）
UnsolResponse	Unsolicited，modem 主动上报（无 serial，走 Indication 类+RegistrantList，第七章）
VoLTE	Voice over LTE，IMS 承载的 4G 语音（第五章）
WakeLock	RIL 请求在途期间持有的电源锁（60 秒超时兜底，RIL.java:292 事件）（第七章）
rild	历史 RIL 守护进程（hardware/ril/libril 参考实现），socket 时代的遗迹（第二章）

排障手册：Telephony 电话栈问题定位

总路线：1. dumpsys 定位状态（telephony.registry 是全系统镜像）→ 2. logcat -b radio 看 modem 对话（RILJ/RILC/GsmCdma 三个 tag，注意电话日志走 radio buffer 而非 main）→ 3. 深入各章（第七章管线/第八章绑定/第九章注册/第十章呼叫）。

四大实战场景（症状/定位/根因/修复）

场景一：无服务（SIM 已插入但状态栏叉号）

症状：logcat -b radio 里反复出现 `RILJ: [UNSL]< UNSOL_RESPONSE_VOICE_NETWORK_STATE_CHANGED` 后紧跟 `GsmCdma[0]: pollState: modemTriggered=true`，但 `handlePollStateResultMessage: CS cellular` 打出的 `mVoiceRegState=1(OUT_OF_SERVICE)`；状态栏信号叉号，紧急呼叫可用或不可用。

定位命令：`adb shell getprop gsm.sim.state`（READY/ABSENT/PIN_REQUIRED 三分支）；`adb shell dumpsys telephony.registry | grep -E "mServiceState|mVoiceRegState|mDataRegState"`；`adb logcat -b radio | grep -E "Registration denied|rejectCause|RILJ: \[UNSL\]<`

UNSOL_RESPONSE_VOICE_NETWORK_STATE_CHANGED"。

根因：按 getprop 分流——ABSENT 是卡槽接触/卡故障；READY 但 out-of-service 看 rejectCause（如 13=漫游不允许、3=禁用小区）；或 radio off（飞行模式未恢复，SST mDesiredPowerState 为 false）。

修复：ABSENT→查卡槽与 SIM 硬件；注册被拒→核对 rejectCause 与运营商白名单/频段支持（carrier config 的 band 过滤）；radio off→检查 AIRPLANE_MODE_ON 与 ENABLE_CELLULAR_ON_BOOT 设置（SST 构造函数读这两项，:688 附近）。

场景二：数据不通（信号满格但上不了网）

症状：`logcat -s DcTracker DataConnection` 出现 `onDataSetupComplete: error apn=xxx, cause=` 或 `DataConnection: DcInactiveState: EVENT_CONNECT initConnection failed`；拨号成功但套接字无路由。

定位命令：`adb shell dumpsys telephony.data`（APN 配置与数据连接状态）；`adb shell dumpsys connectivity | grep -A3 TELEPHONY`（NetworkAgent 评分与 VALIDATED）；`adb shell content query --uri content://telephony/carriers --where "apn='xxx'"`（查 telephony.db 的 APN 记录）。

根因：APN 名/认证错（cause 29/27）；DcTracker 重试退避中（RetryManager 指数退避）；DataCallResponse cause 码（3GPP TS 24.301 拒绝）；数据开关/漫游限制（DataEnabledSettings）。

修复：核对 carrier config 与 APN 表（尤其 mvno 匹配）；看 DataConnection 状态机卡在 DcInactiveState 还是 DcActiveState 的 EVENT_LOST_CONNECTION；按 cause 码对症（auth failure 修密码、被网络拒绝联系运营商）；最后确认 ConnectivityService（书10）侧 NetworkAgent 有没有被评分踢下线。

场景三：呼叫建立失败（拨出即断）

症状：`logcat -b radio` 里 `RILJ: [12]> DIAL <hidden> ...` 之后没有对应 `[12]< DIAL` 应答，或出现 `processResponse: Unexpected response! serial: 12`；Dialer 侧 `CallStateException` 或 `onDisconnect: cause` 日志。

定位命令：`adb shell dumpsys telecom`（Telecom 的 Call 记录与 DisconnectCause）；`adb shell dumpsys telephony.registry | grep mCallState`；`adb logcat -b radio | grep -E "DIAL|CALL_STATE_CHANGED|DriverCall|canDial"`。

根因：checkForDialIssues 拒绝（radio off、通话数满）；modem 返回 GENERIC_FAILURE（运营商侧拒绝/欠费）；三方通话 SWITCH 未落地即 DIAL（10.1 的 500ms 时间窗失手）；号码改写后成 INVALID_NUMBER。

修复：先看 CallStateException 消息定位 canDial 检查项；modem 拒绝按 DisconnectCause 与 cause 码查运营商侧；多路通话抓 AT+CHLD 与 DIAL 的相对时序（对比两条 RILJ 时间戳）；号码问题回查 convertNumberIfNecessary 的 carrier 改写规则。

场景四：rilj 断开重连（HAL 崩溃循环）

症状：`logcat -b radio` 刷屏 `getRadioServiceProxy: ServiceProxy getService/setResponseFunctions: android.os.DeadObjectException`、`EVENT_AIDL_PROXY_DEAD`、`WAKE_LOCK_TIMEOUT mRequestList=N`；随后整段 `[xxx]< RADIO_NOT_AVAILABLE` 应答风暴。

定位命令：`adb logcat -b radio | grep -E "DeadObjectException|PROXY_DEAD|resetProxyAndRequestList|RADIO_NOT_AVAILABLE"`；`adb shell dumpsys telephony.registry | grep mRadioState`；`adb shell ls /data/tombstones/`（vendor HAL crash 现场）。

根因：vendor radio HAL 进程 crash（modem 重启、OOM kill、厂商固件缺陷）；ServiceManager 中服务消失导致版本瀑布逐级 NoSuchElementException；极少数是 SELinux/权限变更。

修复：RIL 会按第八章自愈——死亡通知触发 resetProxyAndRequestList，在途请求回 RADIO_NOT_AVAILABLE，上层状态机重 poll，下次请求重绑；若循环崩溃则查 tombstone 与 HAL 版本（AIDL 2.0 优先、HIDL 1.6→1.0 瀑布是否都被厂商实现接住）；恢复期间禁止上层缓存 RADIO_NOT_AVAILABLE 为永久失败。

深度实战：Telephony的排障场景分析

场景：无信号但 SIM 已插入

1. dumpsys telephony.registry → 看 mServiceState
2. logcat -s RILJ → 检查 RIL 通信是否正常
3. getprop gsm.sim.state → SIM 状态（READY/ABSENT/PIN_REQUIRED）
4. 检查 modem 是否上报网络注册状态
5. 修复：查 APN/频段/运营商策略

Telephony 核心速查表

机制	一句话	源码锚点	所在册
源码位置	frameworks/opt/telephony/（不在 frameworks/base！）	opt/telephony	本册
RIL	RIL:108 socket 模型（比 Binder 更早）	RIL.java:108	本册
GsmCdmaPhone	:141 extends Phone（2G/3G/4G 合一）	GsmCdmaPhone.java	本册
PhoneFactory	:66 按 modem 类型创建实例	PhoneFactory.java:66	本册

机制	一句话	源码锚点	所在册
七跳拨号	Dialer→Telecom→ConnectionService→Phone→RIL→rild→modem		本册
APN	DataNetworkController→NetworkAgent→CS（书10）	telephony/	本册
IMS/VoLTE	ImsPhone 走 SIP+RTP（IP 域打电话）	ImsPhone	本册